`p_loop` as the Next Prompt-Profile Objective

2026-03-22

Bottom line. The next prompt-profile probe should predict direct prompt-level loop probability,

$$p_{\text{loop}}(x) = \mathbb{E}_r \left[1\{\text{rollout } r \text{ loops}\}\right],$$

from prompt-prefill activations under one fixed model and decode policy. The best backup objective is `mean_relative_length = E[L / E]`. Keep `p_cap` diagnostic-first, not as the main head.

Why this changed. The first real `GPQA` prompt-profile pilot ruled out the old high-tail headline target. On the saved `48`-prompt archive, `s_0.9` collapsed exactly to `p(cap-hit)`, and the trained probe was essentially constant-baseline quality. A same-archive `mean_relative_length` retrain was directionally better, but it still lost to trivial baselines on MSE/MAE at that pilot scale.

Why `p_loop` is the right next scalar. The cross-dataset rollout-stat bundle says the failure mode is fundamentally a terminal-statistics problem:

- loops are much longer than ordinary wrong rollouts, which are longer than correct ones;
- looped rollouts are much less accurate;
- `P(loop | max-length-hit)` is effectively `1.0` across all five datasets.

That makes `p_loop` the cleanest direct prompt-level summary of the behavior we actually care about: "under this prompt and this fixed policy, how often do I enter a looping terminal regime?"

Why not the obvious alternatives.

- **`majority_s_0.5`.** It does show real activation signal on the corrected `GPQA` pilot, but with `n=4` it is a lossy thresholded view of the same rollout counts. Counts `0`, `1`, and `2` all map to the same negative label.
- **`s_t`.** High-threshold tails are too close to cap-hit on the current long-cap surface, while lower-threshold tails buy density by mixing in more non-loop long answers.
- **`mean_relative_length`.** It is the best no-plumbing fallback, but it is still a proxy that mixes correct long reasoning, wrong long reasoning, and looped long reasoning.
- **pure cap risk.** `p_cap` is sharp but too narrow. Many loops do not hit the cap, especially on `GPQA` and `MMLU-Pro`.

The key GPQA leakage check. On the saved `GPQA` prompt-profile archive, `p_loop` is also the least prompt-length-correlated of the obvious scalar targets:

Target	Spearman with prompt length
`p_loop`	0.297
`mean_relative_length`	0.375
`p_cap`	0.346
`s_0.5`	0.341
`s_0.6`	0.373
`s_0.9`	0.346

So the direct loop-rate target is not only more aligned than the length-style proxies; on the current `GPQA` slice it is also less length-leaky.

Concrete next run.

1. **Split.** Prompt-disjoint, in-distribution only, read per dataset rather than pooled first.
2. **Input.** Last prompt-token prefill activations, comparing:
   - one selected-layer MLP probe;
   - one per-layer MLP ensemble with score aggregation by `mean_prob`.
3. **Target.** `-target-kind probability -profile-target p_loop`.
4. **Backup.** `-target-kind regression -profile-target mean_relative_length`.
5. **Baselines.** Constant, prompt-token-count only, effective-budget only, and prompt-token-count plus effective-budget together.

Evaluation bundle. For the main `p_loop` head, use the probability-target bundle already in the repo:

- Brier;
- MAE;
- Spearman;
- top-`10%` and top-`20%` capture of realized loop mass.

Then attach downstream diagnostics on the same ranked prompts:

- observed `p_cap` in the top-risk buckets;
- observed correctness in the top-risk buckets;
- comparison against prompt-length-only and effective-budget-only controls.

Implementation note. This does not require a new data pipeline. The prompt-profile aggregator already computes `p_loop` for every prompt. The useful code change is just to expose that existing field as a trainable probability target in the dataset builder so the next run can call `build_probe_dataset.py -target-kind probability -profile-target p_loop` directly.

Recommendation. Stop spending the single-head prompt-profile path on thresholded surrogates first. Ship `p_loop`. If it still fails to beat the leakage baselines on the larger prompt-disjoint runs, fall back to `mean_relative_length` rather than another `s_t` variant.